

Evaluating Multi-Agent Systems

This week's evaluation anchor — trajectory, cost, and regression, not just a single response

Source: <https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators>

Two evaluation angles

System evaluation

- Examines the end-to-end outcome of the whole workflow
- Applies to the orchestrator, or the final agent responsible for completion

Process evaluation

- Examines the quality and efficiency of each step
- Focuses on the tool calls executed along the way

System evaluation: the outcome

Evaluator	Answers
Task Completion *(preview)*	Did the agent complete the requested task with a usable deliverable?
Task Adherence *(preview)*	Did the agent follow its assigned rules and constraints?
Task Navigation Efficiency	Did the agent take an efficient path, compared to a known-good sequence?
Intent Resolution *(preview)*	Did the agent correctly identify what the user actually wanted?
Customer Satisfaction *(preview)*	How satisfied would a user be, across helpfulness, clarity, resolution?

Task Navigation Efficiency in code

```
{  
  "type": "azure_ai_evaluator",  
  "name": "task_navigation_efficiency",  
  "evaluator_name": "builtin.task_navigation_efficiency",  
  "initialization_parameters": {"matching_mode": "in_order_match"},  
  "data_mapping": {  
    "actions": "{{item.actions}}",  
    "expected_actions": "{{item.expected_actions}}",  
  },  
}
```

matching_mode is exact_match (order and content), in_order_match (extra steps allowed, order preserved), or any_order_match (extra steps allowed, any order) — the same "extras OK" trade-off Day 3's tool_calls_present made for a single agent.

Process evaluation: the steps

Evaluator	Answers
Tool Call Accuracy	Right tool calls, correct parameters, no redundancy?
Tool Selection	Correct and necessary tools chosen, nothing extra?
Tool Input Accuracy	Are all tool-call parameters correct — groundedness, type, format, required/unexpected?
Tool Output Utilization	Did the agent actually use the tool's result correctly downstream?
Tool Call Success	Did the tool calls succeed, or hit technical errors?

What "trajectory" means here

- A trajectory is the ordered sequence of actions taken across the whole workflow — every agent's tool calls, in the order they happened
- For the lab: Planner's decomposition, Retriever's lookups, Critic's accept/loop decision, and (if triggered) the loop back through Planner again
- Task Navigation Efficiency's actions field is exactly this: a list of `function_call` messages, compared against a known-good `expected_actions` list

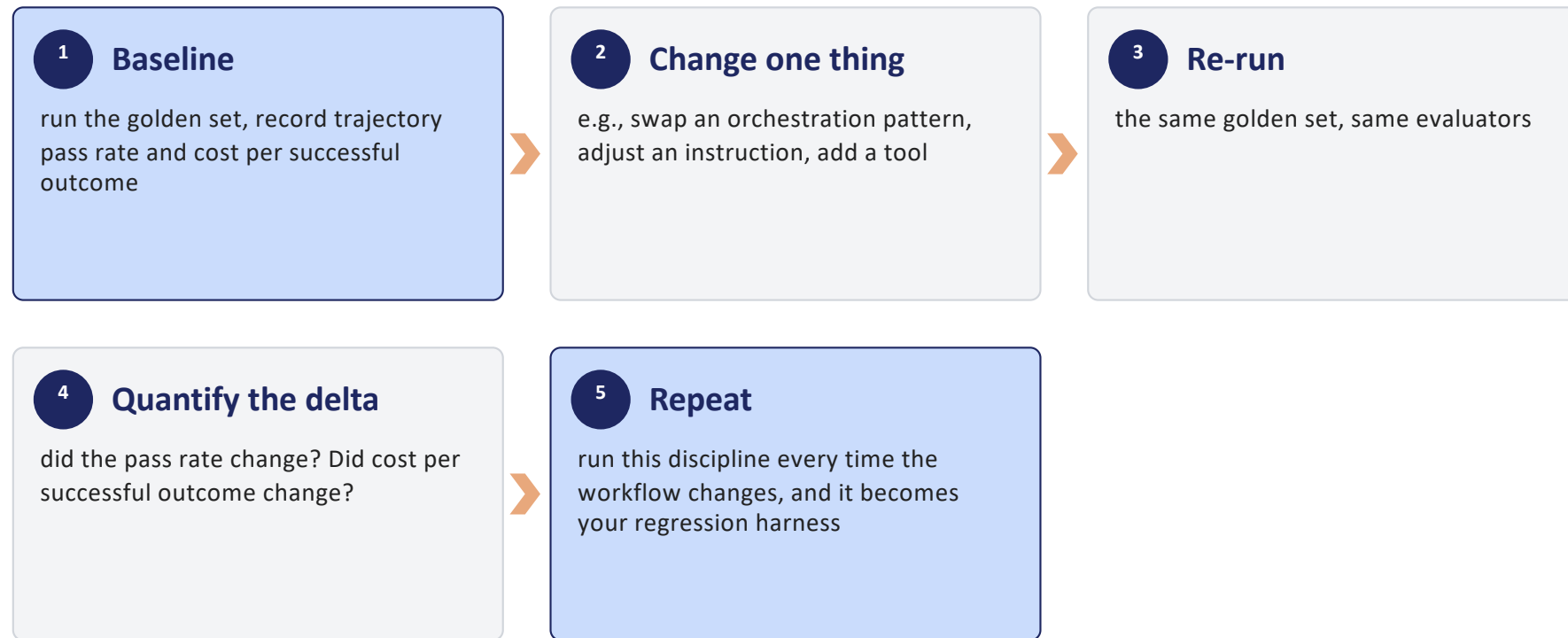
Building a golden set for a workflow

- Same discipline as Day 3 Part E, at workflow scale: real queries, expected outcomes, not synthetic passes
- The lab's own golden set: 8 cases across 4 branches — small on purpose, since every case runs against all 3 parts × 3 repetitions (~72 workflow runs already)
- Cover the workflow's real branches: grounded (clean single-document lookup), revision (spans documents, needs a Critic-triggered loop), no_retrieval (general knowledge, no tools), and partial (the corpus answers half — naming the gap is correct, inventing the rest is a failure)

Cost per successful outcome

- The framework emits `gen_ai.client.token.usage` (a histogram metric) and `gen_ai.usage.input_tokens/output_tokens` (span attributes) for every model call, automatically
- Cost per successful outcome = sum of tokens across every agent's model calls in a run, divided by the count of golden-set cases that actually passed
- A 3-agent workflow that succeeds cheaply beats a 3-agent workflow that succeeds expensively — this metric is how you tell them apart, not just pass/fail

The eval → change → re-eval loop



D E M O

~4 min

Trajectory, cost, and the eval → change → re-eval loop

Run the SAME 2-agent workflow twice: BEFORE, the researcher's instructions are vague and it may skip its one tool — process eval and system eval both at risk of failing. AFTER, one sentence is added requiring the tool call — nothing else changes. Compare trajectory (process eval), the final answer (system eval), and total tokens (cost) across the two runs: this IS the loop the slide just named, run live.

Runbook: [demos/day4/module-5-demo-1-trajectory-and-cost.md](#)

Repetitions still matter at workflow scale

- Every agent's model call is a fresh source of nondeterminism — a 3-agent workflow has three (or more) independent chances to vary
- `num_repetitions` still applies: run each golden-set case multiple times, observe the pass-rate distribution
- Don't invent a universal pass threshold — set acceptance criteria from your own risk tolerance and observed variance, same discipline as Day 3

Takeaways

- ✓ You evaluate the workflow's end-to-end outcome (System evaluation) separately from the quality of each step (Process evaluation).
- ✓ Task Navigation Efficiency generalizes Day 3's `tool_calls_present/tool_call_args_match` to a full multi-agent trajectory.
- ✓ Cost per successful outcome comes from the framework's own token-usage telemetry, not a separate cost API.
- ✓ Your "regression harness" is the eval → change → re-eval loop, run consistently every time the workflow changes.
- ✓ Repetitions matter more, not less, with multiple agents — more model calls means more sources of variance.